

Evaluating Graph-Based and Traditional Machine Learning Models for Airbnb Price Prediction



Ethan Cohen¹ Haotian (Owen) Cui¹ Matthieu Hautsch²

{ethancoh, owencui, matthaut}@stanford.edu

¹Institute of Computational and Mathematical Engineering, Stanford

²Civil & Environmental Engineering, Stanford

Introduction

- **Motivation:** Accurate pricing ensures **fair revenue for hosts**, **fair costs for guests**, and **consistent valuations for platforms**.
- **Goal:** Understand which factors drive Airbnb listing prices in San Francisco and how accurately machine learning models can predict **log-price**.
- **Inputs:** Listing-level attributes, spatial information, and review-based indicators.
- **Models:** KNN baseline, LASSO regression, XGBoost, a multilayer perceptron (MLP), and an inductive Graph Sample and Aggregate (GraphSAGE) Neural Network.
- **Result:** Best model, which is an **XGBoost + SBERT embeddings**, achieves a test MAE of approximately 0.312.

Data & Features

Data:

- We use the San Francisco listings dataset from the *Inside Airbnb* open-data project [1], which initially contains over 7,000 listings with tabular attributes, text descriptions, geographic coordinates, and ground-truth prices. We apply a log transform to the price variable to reduce right skewness and stabilize variance.
- After restricting to short-term stays and performing data cleaning (removing missing values and capping extreme outliers), the final dataset contains **5,182 listings**.
- To evaluate generalization under distribution shifts, we use a **random 80/20 split** (D1) and a **geo-clustered split** (D2) where listings are grouped into 20 k -means clusters and 5 entire clusters are held out as the test set (Figure 1); corresponding evaluation protocols **S1–S3** are summarized in Table 1.

Name	Definition	Dataset
D1	Random 80/20 train–test split; allows spatial leakage.	4,145 / 1,037
D2	Geo-aware split based on spatial clusters; prevents neighborhood leakage.	4,402 / 780
S1	Standard 5-fold Cross Validation with random shuffling (optimistic baseline).	D1 only
S2	Geo-clustered 5-fold Cross Validation without SBERT embeddings.	D2 only
S3	Geo-clustered 5-fold Cross Validation with SBERT embeddings (final evaluation setup).	D2 only

Table 1. Definitions of dataset splits and evaluation settings

Features:

- **Raw Features:** include *description*, *neighborhood*, *accommodates*, *room type*, *bathrooms*, *bedrooms*, *beds*, *amenities*, *host/review indicators*, *availability*, and *latitude/longitude*.
- We engineer comfort-related features such as **bedrooms_per_guest**, **bathrooms_per_guest**, and **amenity_count**, plus a **geo_cluster** label computed using k -means on latitude/longitude.
- Description text is embedded using the SBERT model **all-MiniLM-L6-v2** [3], producing 384-dimensional semantic vectors for each listing.
- These combined features capture listing capacity, amenities, quality, location, and semantic cues, enabling both classical models and the Graph Neural Network (GNN) to learn spatial and textual signals that influence log-price.

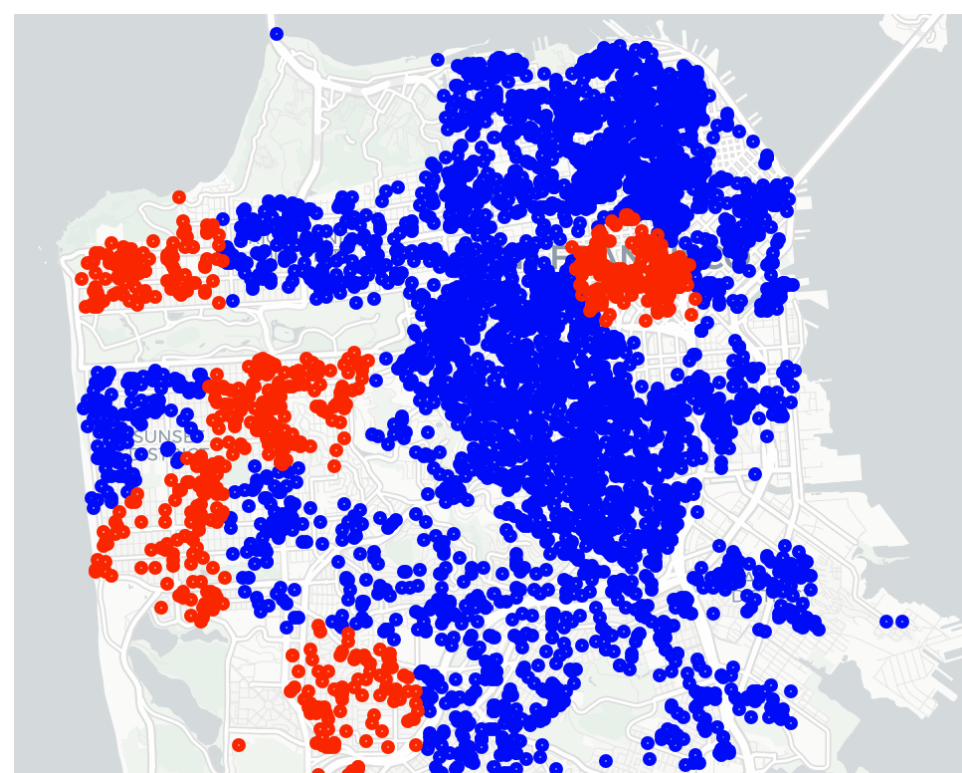


Figure 1. Geographical train (blue) - test (red) split of San Francisco Airbnb listings

Models

We compare a set of tabular baselines (KNN, LASSO, MLP, XGBoost) with a graph-based model.

1. **KNN (baseline):** We concatenate tabular features and SBERT embeddings and standardize them. We run k -nearest neighbors regression with Minkowski distance

$$d(\mathbf{z}, \mathbf{z}_i) = \left(\sum_j |z_j - z_{ij}|^p \right)^{1/p},$$

and predict log-price by a distance-weighted average over the k closest neighbors.

2. **LASSO regression:** We fit a linear model for log-price by solving

$$\min_{\beta} \sum_{i=1}^n (y_i - x_i^{\top} \beta)^2 + \lambda \|\beta\|_1,$$

so the ℓ_1 penalty enforces sparsity and performs implicit feature selection. On S3, LASSO sets about 83% of SBERT text coefficients exactly to zero, keeping only a compact set of informative textual dimensions.

3. **MLP:** The MLP maps inputs x to log-price via three hidden layers with ReLU activations and dropout,

$$h_1 = \text{ReLU}(W_1 x + b_1), \quad h_2 = \text{ReLU}(W_2 h_1 + b_2), \quad h_3 = \text{ReLU}(W_3 h_2 + b_3), \quad \hat{y} = w^{\top} h_3 + c.$$

4. **XGBoost (gradient-boosted trees):** We model log-price with an ensemble of shallow regression trees where each tree is trained to correct residuals from previous trees. XGBoost uses shrinkage, subsampling, and ℓ_2 regularization to prevent overfitting.
5. **Graph Neural Network:** We build a graph $G = (V, E)$ where each node is a listing. Edges connect listings via k_{spatial} -NN in haversine distance [4] on (lat, lon) and k_{text} -NN in cosine distance on SBERT embeddings. A two-layer inductive GraphSAGE network [2] updates node embeddings by aggregating neighbor information

$$h_i^{(\ell+1)} = \text{ReLU} \left(W_{\text{self}}^{(\ell)} h_i^{(\ell)} + W_{\text{neigh}}^{(\ell)} \text{mean}_{j \in \mathcal{N}(i)} h_j^{(\ell)} + b^{(\ell)} \right),$$

and a final linear layer outputs the log-price. We train inductively on training geo-clusters and evaluate on separate graphs for held-out clusters.

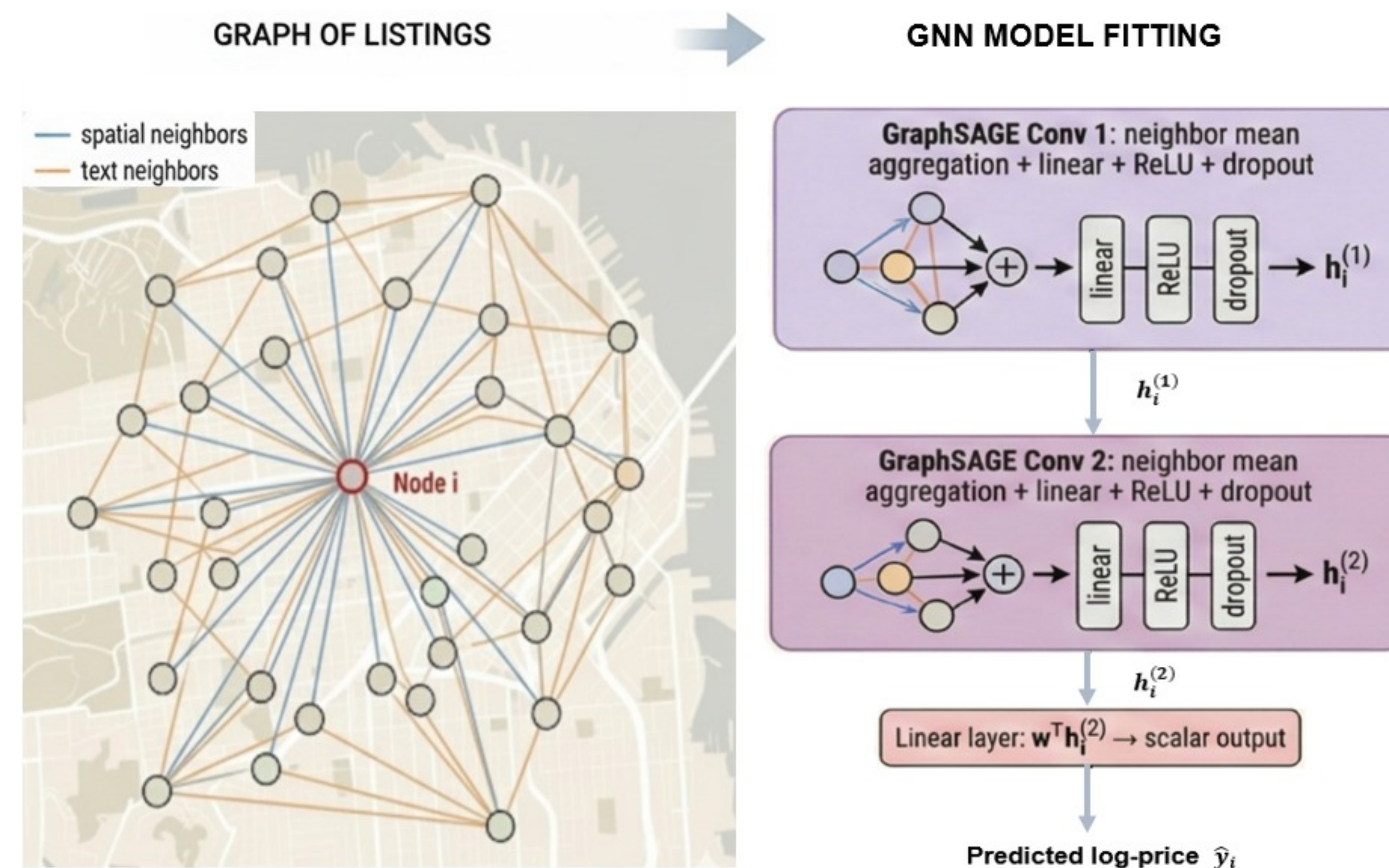
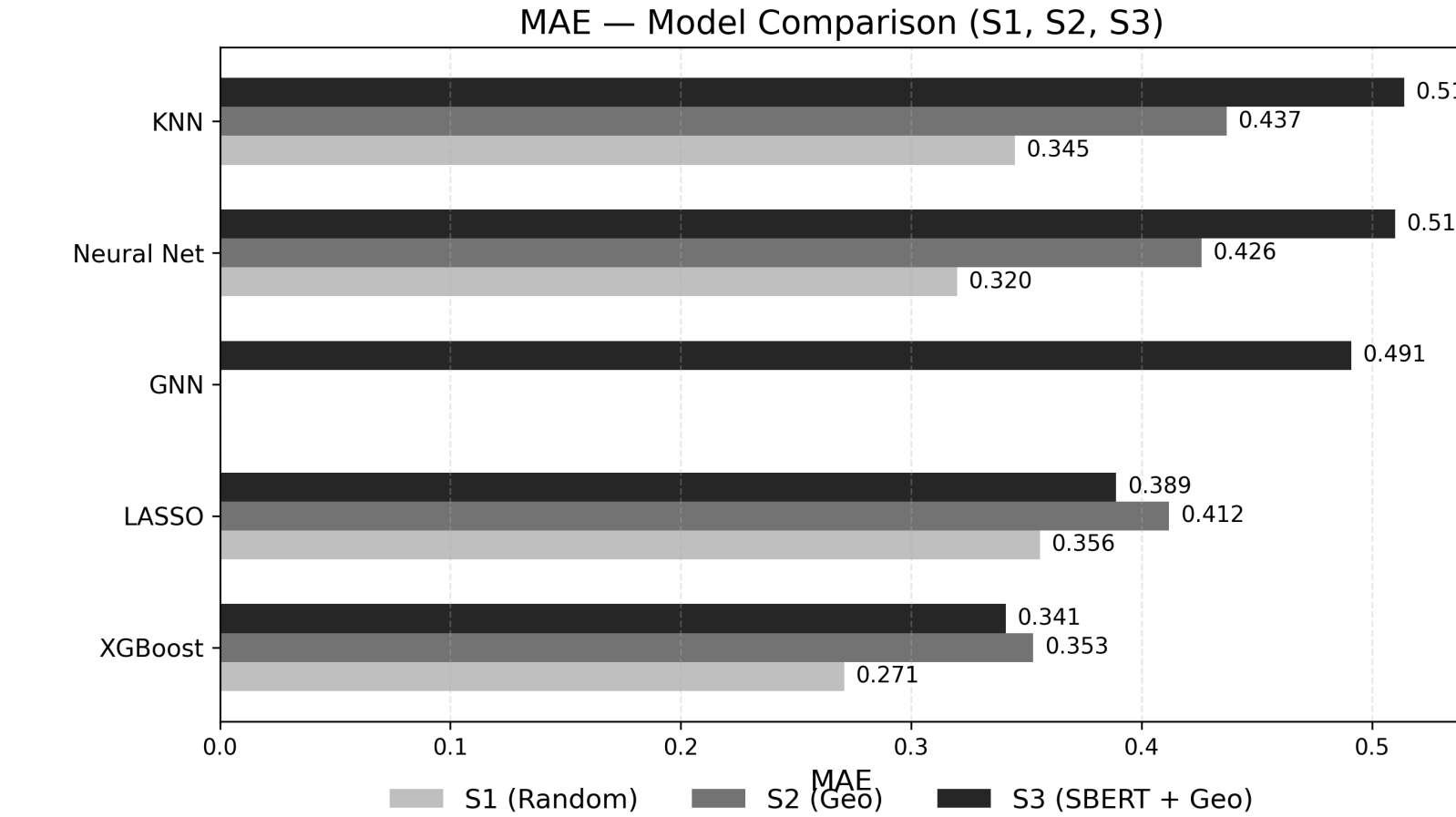


Figure 2. GraphSAGE Pipeline: Spatial/text KNN graph (left) and two-layer GraphSAGE architecture (right)

Results & Discussion

Model Performance



Model	Train MAE	Test MAE
XGBoost + SBERT	0.120	0.312
GraphSAGE GNN	0.315	0.395

Figure 4. Train and test MAE on held-out geo clusters for our best model XGBoost and our main model GraphSAGE GNN.

Note: GraphSAGE is only evaluated under S3 (with SBERT text embeddings).
Figure 3. 5-fold CV MAE for all models under S1, S2 and S3

Prediction Quality

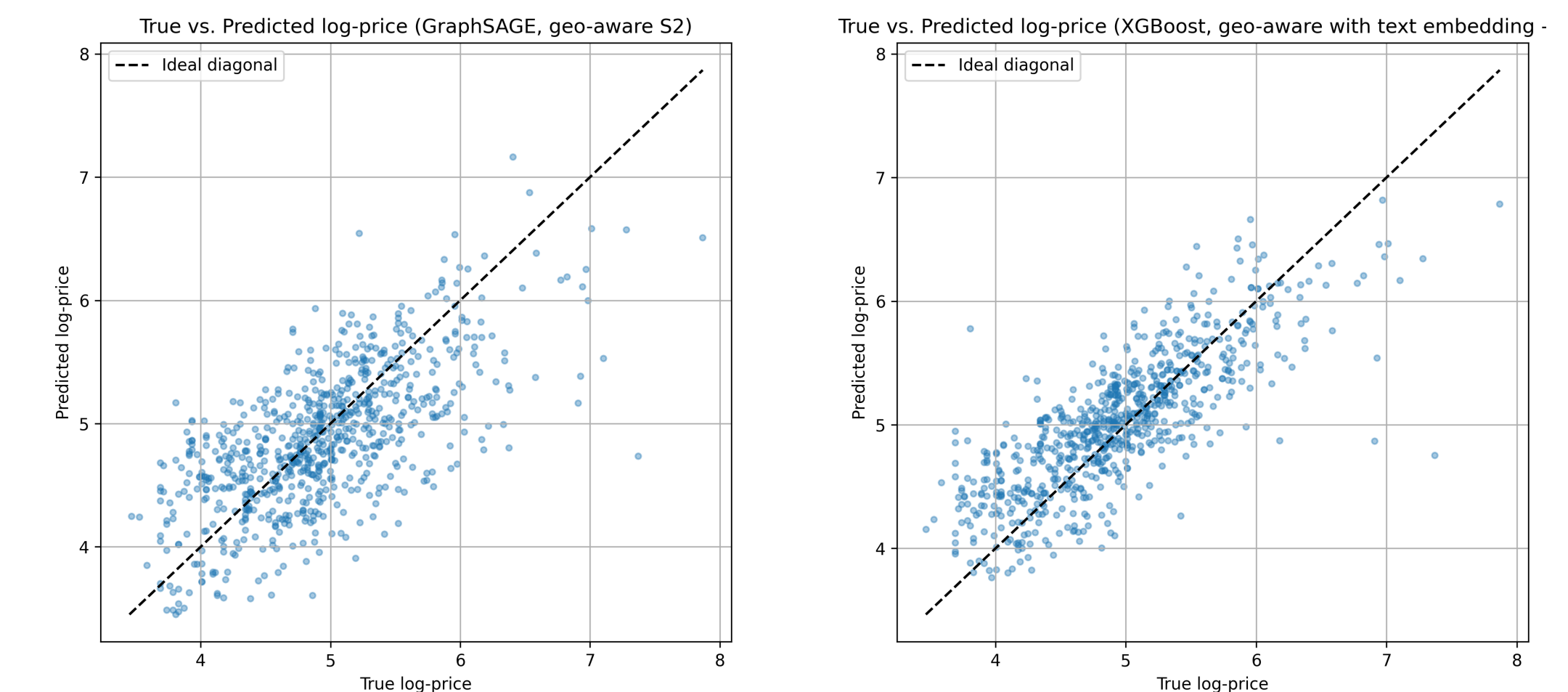


Figure 5. True vs. Predicted log-price for GNN and XGBoost

Discussion

- Errors increase when moving from random splits (S1) to geo-aware S2/S3, confirming that **standard CV overestimates performance** by leaking spatial information, with MAE typically **20–25% lower** under S1 than under geo-aware splits.
- SBERT text features hurt simple models but help regularized ones like **LASSO** and **XGBoost**, which reach the best MAE.
- **XGBoost** attains the lowest test MAE (0.312) but shows stronger overfitting, whereas **GraphSAGE** (0.395 MAE) has a smaller train–test gap, reflecting a smoother inductive bias.

Future Research

- **Richer data across cities and time:** add seasonality, events, and multi-city data to test how well models transfer to new neighborhoods and markets.
- **GNN–XGBoost hybrid:** use a GraphSAGE encoder to build neighborhood-aware features, then feed them into XGBoost instead of raw tabular inputs.
- **Impact of graph structure:** vary how we build the graph (number of neighbors, connection rules, GNN depth) to see in which settings GNNs actually outperform boosted trees.

References

- [1] Inside airbnb.
<http://insideairbnb.com/get-the-data/>. Accessed 2024.
- [2] William L Hamilton, Rex Ying, and Jure Leskovec.
Inductive representation learning on large graphs.
arXiv preprint arXiv:1706.02216, 2017.
- [3] Nils Reimers and Iryna Gurevych.
Sentence-bert: Sentence embeddings using siamese bert-networks.
arXiv preprint arXiv:1908.10084, 2019.
- [4] C. C. Robusto.
The cosine-haversine formula.
The American Mathematical Monthly, 64(1):38–40, 1957.